

Nama : Daniswara Zakiy Anhari
Kelompok : Aquamarine

PROBLEM 1 – THE LAST ASTRONAUTS

Pseudocode

PROGRAM TheLastAstronauts

BEGIN

 DECLARE n = 0

 DECLARE k = 0

 PRINT "Masukkan jumlah astronaut (n): "

 INPUT n

 PRINT "Masukkan langkah eliminasi (k): "

 INPUT k

 IF n > 1000

 THEN

 PRINT "Maksimal astronaut adalah 1000!"

 END

 END IF

 DECLARE array astronauts[1000]

 FOR i FROM 0 UNTIL n - 1

 DO

 SET astronauts[i] = i + 1

 END FOR

 DECLARE currentCount = n

 DECLARE index = 0

 PRINT "Urutan Eliminasi Astronauts: "

 WHILE currentCount > 1

 DO

 SET index = (index + k - 1) MOD currentCount

 DECLARE eliminated = astronauts[index]

 PRINT eliminated

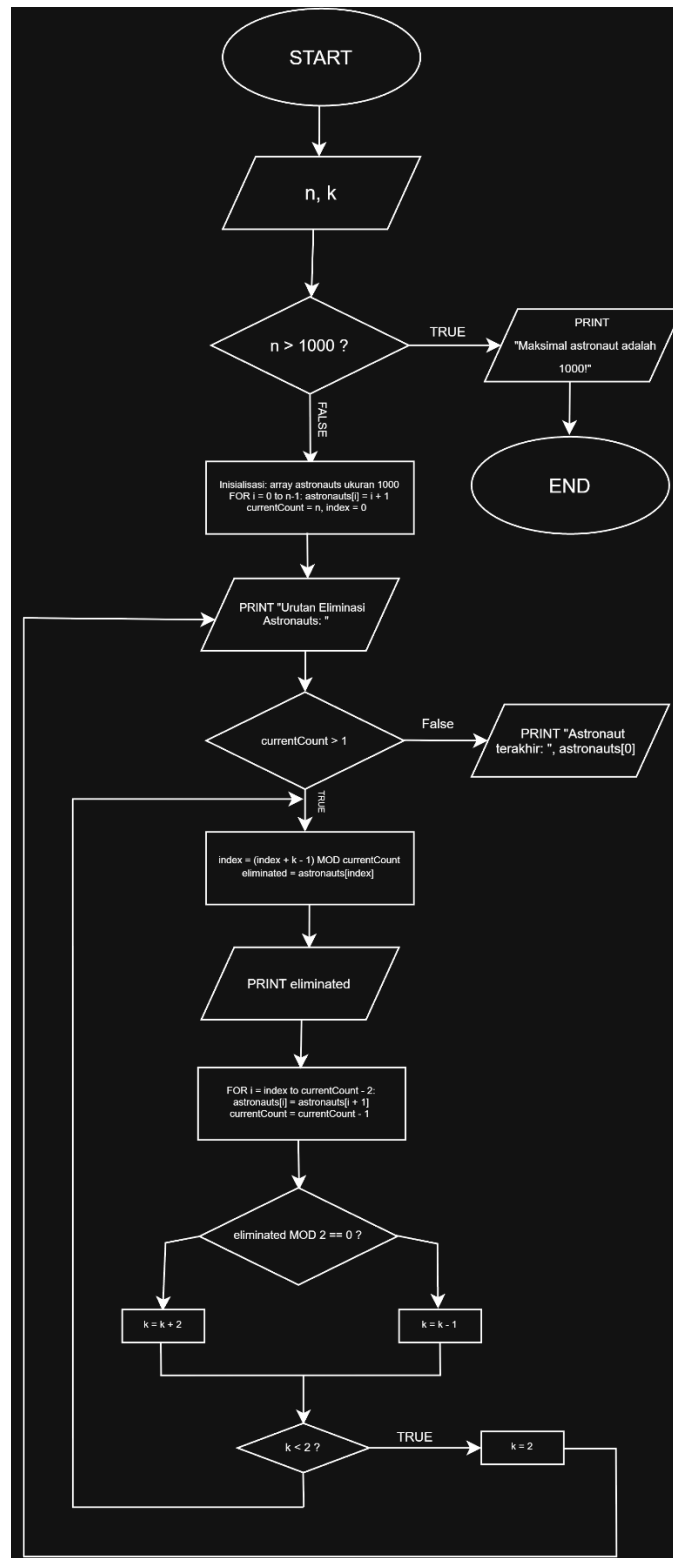
 FOR i FROM index UNTIL currentCount - 2 DO

```
    SET astronauts[i] = astronauts[i + 1]
END FOR

SET currentCount = currentCount - 1
IF eliminated MOD 2 == 0 THEN
    SET k = k + 2
ELSE
    SET k = k - 1
END IF

IF k < 2 THEN
    SET k = 2
END IF
END WHILE
PRINT "Astronaut terakhir: ", astronauts[0]
END
```

Flowchart



PENJELASAN

Program dimulai dengan inisialisasi, di mana program menerima masukan berupa jumlah keseluruhan astronot (n) dan nilai hitungan eliminasi awal (k). Program memeriksa apakah jumlah astronot tidak lebih dari 1000 orang. Setelah itu, program membuat array bernama `astronauts` dan mengisinya dengan nomor 1 sampai n untuk menandai setiap astronot. Proses eliminasi utama program menjalankan dengan perulangan `WHILE` yang terus berjalan selama `currentCount` lebih dari satu.

Dalam perulangan, program menghitung posisi astronot yang harus tereliminasi menggunakan rumus $(\text{index} + k - 1) \text{ MOD } \text{currentCount}$. Rumus ini penting supaya hitungan bersifat melingkar; kalau hitungan melewati akhir barisan, posisi akan kembali ke awal. Setelah astronot pada posisi tersebut dihapus, program memindahkan elemen kanan ke kiri untuk menutupi ruang kosong, lalu mengurangi total `currentCount`.

Sesuai aturan misi, program memperbarui nilai k untuk iterasi berikutnya: jika nomor astronot yang baru gugur genap, program menambah k 2; jika ganjil, program mengurangi k 1. Program juga memastikan nilai k tidak kurang dari 2, dan jika kurang, program set k menjadi 2. Siklus eliminasi dan pembaruan k ini terus berulang sampai array hanya ada satu elemen di indeks 0, lalu program mencetak astronot terakhir yang bertahan.

PROBLEM 2 – Alien-In-The-Middle

Pseudocode

PROGRAM alienInTheMiddle

FUNCTION getStringLength(str)

 DECLARE length = 0

 WHILE str[length] != '\0' DO

 SET length = length + 1

 END WHILE

 RETURN length

END FUNCTION

FUNCTION encryptAlien(message)

 IF message[0] == '\0' THEN

 RETURN

 END IF

 PRINT message[0]

 DECLARE length = CALL getStringLength(message)

 FOR i FROM 1 UNTIL length - 1 DO

 DECLARE previousValue = (nilai ASCII message[i - 1]) - (nilai ASCII 'A') + 1

 DECLARE currentValue = (nilai ASCII message[i]) - (nilai ASCII 'A')

 DECLARE newValue = (previousValue + currentValue) MOD 26

 DECLARE newChar = karakter dari (nilai ASCII 'A' + newValue)

 PRINT newChar

 END FOR

 PRINT baris baru ('\n')

END FUNCTION

BEGIN

 DECLARE array message[1000]

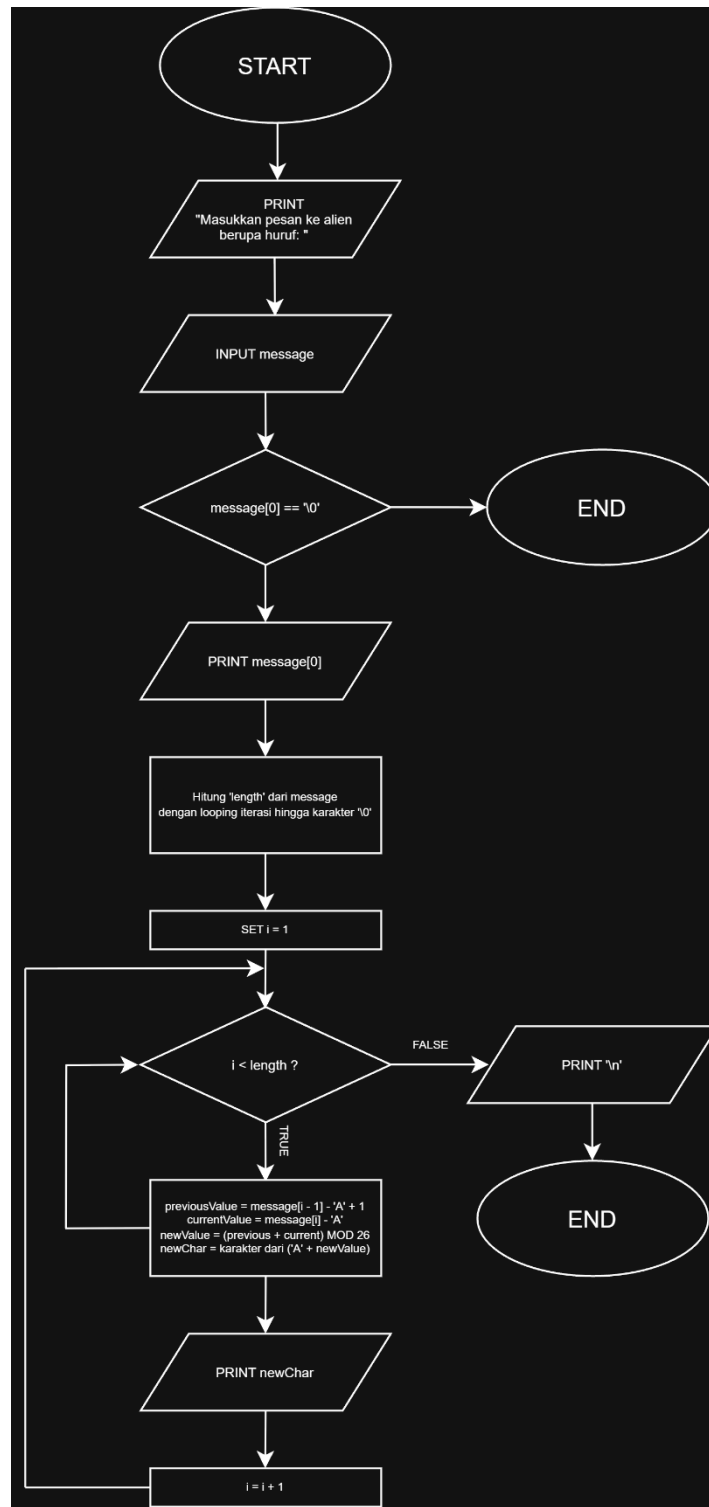
 PRINT "Masukkan pesan ke alien: "

 GET message

 CALL encryptAlien(message)

END

Flowchart



PENJELASAN

Program ini dirancang dengan memecah tugas ke dalam fungsi-fungsi sendiri, di mana eksekusi dimulai dengan menerima masukan pesan dari pengguna berupa array karakter, lalu mengirimkannya ke fungsi `encryptAlien`. Di dalam fungsi enkripsi tersebut, program pertama-tama memastikan bahwa pesan tidak kosong. Jika valid, program akan langsung mencetak huruf pertama dari pesan tanpa melakukan perubahan sama sekali, sesuai dengan aturan sistem sandi di mana huruf pertama memang tidak bergeser. Selanjutnya, program menghitung total panjang karakter pesan dengan memanggil fungsi kustom `getStringLength` (yang terus menghitung hingga menemukan karakter null terminator `\0`). Panjang pesan ini digunakan sebagai batas perulangan untuk memproses sisa huruf lainnya.

Inti dari algoritma penyandian rahasia ini terletak pada perulangan yang mengeksekusi huruf kedua hingga huruf terakhir. Pada setiap iterasinya, program mengambil huruf yang tepat berada di posisi sebelumnya, lalu mengonversi nilai ASCII-nya menjadi nilai angka alfabetis berbasis 1 (di mana A=1, B=2, hingga Z=26) yang bertindak sebagai jarak pergeseran. Jarak ini kemudian ditambahkan ke posisi huruf yang sedang diproses saat ini. Untuk memastikan huruf yang digeser tidak keluar dari rentang alfabet (misalnya jika hasil pergeseran melewati huruf Z), program menggunakan operasi sisa bagi atau modulo 26 ($\text{MOD } 26$) agar hitungan kembali memutar dari awal (huruf A). Hasil akhir dari perhitungan tersebut dikonversi kembali menjadi wujud karakter (huruf baru) dan dicetak langsung ke layar, hingga membentuk pesan rahasia yang utuh pada akhir perulangan.